A Joint Approach to Local Updating and Gradient Compression for Efficient Asynchronous Federated Learning

Jiajun Song[1], Jiajun Luo[2], Rongwei Lu[1], Shuzhao Xie[1], Bin Chen[3], and Zhi Wang[1]*

[1] Shenzhen International Graduate School, Tsinghua University; Tsinghua-Berkeley Shenzhen Institute, Tsinghua University
[2] Department of Computer Science and Engineering, Southern University of Science and Technology
[3] School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen
sjj23@mails.tsinghua.edu.cn, 12012023@mail.sustech.edu.cn, lrw21@mails.tsinghua.edu.cn, xiesz99@gmail.com, chenbin2021@hit.edu.cn, wangzhi@sz.tsinghua.edu.cn

Abstract. Asynchronous Federated Learning (AFL) confronts inherent challenges arising from the heterogeneity of devices (e.g., their computation capacities) and low-bandwidth environments, both potentially causing *stale* model updates (e.g., local gradients) for global aggregation. Traditional approaches mitigating the staleness of updates typically focus on either adjusting the local updating or gradient compression, but not both. Recognizing this gap, we introduce a novel approach that synergizes local updating with gradient compression. Our research begins by examining the interplay between local updating frequency and gradient compression rate, and their collective impact on convergence speed. The theoretical upper bound shows that the local updating frequency and gradient compression rate of each device are jointly determined by its computing power, communication capabilities and other factors. Building on this foundation, we propose an AFL framework called FedLuck that adaptively optimizes both local update frequency and gradient compression rates. Experiments on image classification and speech recognization show that FedLuck reduces communication consumption by 56% and training time by 55% on average, achieving competitive performance in heterogeneous and low-bandwidth scenarios compared to the baselines.

Keywords: Asynchronous Federated Learning · Gradient Compression · Local Update · Staleness · Joint Optimization

1 Introduction

Asynchronous Federated Learning (AFL) [13,22,32] is a machine learning method where devices train a model together without synchronizing [15]. However, this

* Corresponding author

approach struggles with the varying local training speeds and capacities of devices, as well as low-bandwidth networks, leading to a problem known as *model staleness* [5, 8, 10, 18, 36]. The features of the asynchronous mechanism facilitate the emergence of model staleness due to the heterogeneous update speeds among different devices and communication time delays in low-bandwidth settings. The heterogeneity and limited bandwidth can cause certain devices to fall behind, leading to outdated local gradients that degrade the global model's performance. Solving model staleness is crucial for AFL deployment in real-life scenarios.

Several strategies have been explored to mitigate this issue. Some employ fusion weight methods [13, 26, 32, 37] for adaptive aggregation during global updates based on staleness levels. Mathematical techniques like Taylor expansion and Hessian matrix approximation also mitigate staleness impact [34]. Besides, clustering devices [30, 35] by model staleness levels is also proven to be effective. Some studies [6, 7] focus on adjusting local updating frequency to accelerate convergence, but they overlook communication challenges, particularly in low-bandwidth scenarios where communication is often the bottleneck [29]. To further reduce communication consumption, gradient compression algorithms [2–4, 11, 19, 28] are used to expedite transmission. However, many employ fixed compression rates without fully exploring the correlation between compression rates with other factors. Recent research has delved into this relationship, considering factors like device data volumes and network conditions [1, 9, 20]. This has spurred the development of adaptive gradient compression algorithms, dynamically adjusting compression rates based on these factors.

Existing works on adjusting local updating frequency and compression rate [6,7,13] have been proven effective. However, they do not adequately consider the joint optimization of both local updating and communication. Although beneficial in isolation, the lack of simultaneous consideration of both components may lead to inefficient solutions as these approaches overlook potential interactions between local updating, communication, and convergence speed. For instance, the gradient obtained through more local updates often requires a bigger compression rate to preserve more information. Besides, in either LANs or WANs, transferring a complete model takes $10 - 100\times$ times longer than local updating time. Therefore, considering local updating or gradient compression alone often results in low convergence speed. We need to consider both mechanisms to improve the convergence speed. Though some synchronous FL methods [23,33] have explored the joint optimization of local updating and gradient compression, AFL remains unexplored. Simply migrating synchronous optimization algorithms to AFL poses challenges such as inadequate low-speed device local training.

Our primary objective is to improve convergence speed through joint optimization of local updating and communication. To achieve this, we adjust two key parameters: the local updating frequency, which affects the time of local training, and the gradient compression rate, which influences communication time. However, this approach presents a dilemma. On one hand, high local updating frequency and compression rate lead to significant model staleness, hindering the convergence of the global model; On the other hand, low values for

these parameters cause significant compression errors, impeding the accuracy of the global model. Our experimental motivation reveals that with appropriate settings, convergence occurs 11× faster than with the most inappropriate settings in average.

Therefore, a critical question emerges: **Given a specific AFL system, how can we jointly and adaptively decide the local updating frequency and the compression rate to converge efficiently?** In this paper, we undertake a theoretical analysis to reveal the complex interplay between the two parameters and their impact on convergence speed. Our analysis leads to the derivation of an upper bound for convergence speed based on the two parameters. Based on this upper bound, we introduce an optimization problem tailored to determine these values for the two parameters. Building upon these insights, we introduce **FedLuck**, an AFL framework designed to enhance convergence efficiency by adjusting both local updating frequency and gradient compression rate. The main contributions are summarized as follows:

- We propose FedLuck, a novel AFL framework to enhance convergence efficiency by jointly and adaptively deciding the local updating frequency and gradient compression rate.
- We conduct a theoretical analysis of the convergence speed and derive an upper bound with respect to the local updating frequency and compression rate. We find a **key convergence factor** ϕ, which exerts an impact of the largest magnitude on the convergence speed. We implemented FedLuck which jointly and adaptively decides the local update frequency and the compression rate for each device by minimizing ϕ.
- We extensively evaluated the performance of FedLuck through simulations and testbed experiments. The evaluation results demonstrate that FedLuck achieves significant reductions in communication consumption by 56% and completion time by 55% on average while maintaining competitive performance compared to the baselines.

2 Preliminaries and Problem Formulation

2.1 Federated Learning

This paper focuses on a parameter server (PS) architecture, comprising a centralized PS for global aggregation and a set of N distributed local devices represented as $\mathcal{N}$. The PS maintains a global model $\mathbf{w} \in \mathbb{R}^d$. Each device, represented by $i \in \mathcal{N}$, possesses a model weight $\mathbf{w}_i$ and a local dataset D_i. Within this dataset, there exist $|D_i|$ data samples, expressed as $\xi_i = [\xi_{i,1}, \xi_{i,2}, \cdots, \xi_{i,|D_i|}]$, which are utilized for local training. We define the loss function for each data sample $\xi_{i,j}$ ($j \in [1, |D_i|]$), as $f(\mathbf{w}_i, \xi_{i,j})$, and denote the local loss function of device i as:

$$F_i(\mathbf{w}_i) := \frac{1}{|D_i|} \sum_{j=1}^{|D_i|} f(\mathbf{w}_i, \xi_{i,j}), \tag{1}$$

where the global loss function can be formulated as $F(\mathbf{w}) := \frac{1}{N} \sum_{i \in \mathcal{N}} F_i(\mathbf{w})$.

The target of this system is to train a global model $\mathbf{w}^*$ that minimizes the global loss function:

$$\mathbf{w}^* := \arg\min_{\mathbf{w}\in\mathbb{R}^d} F(\mathbf{w}). \tag{2}$$

In FL, the whole training process can be divided into model broadcasting, local training, and model aggregation. This iterative update process continues for multiple rounds, allowing the global model to be refined and improved over time.

2.2 AFL with Periodic Aggregation

We specifically target AFL with periodic aggregation. The server periodically aggregates the received gradients from devices that have completed their computations, while allowing other devices to continue their local training uninterrupted. The interval between two global rounds remains constant. Once a device completes its local training, it transmits its local gradient to the server.

Specifically, the entire training process comprises a total of T global rounds. Within each global round, every local device i undertakes k_i local iterations. At each local iteration j, ranging from 0 to $k_i - 1$, local device i updates its local model following the rule:

$$\mathbf{w}_i^{t,j+1} = \mathbf{w}_i^{t,j} - \eta_l \nabla F_i(\mathbf{w}_i^{t,j}, \xi_i^{t,j}), \tag{3}$$

where η_l is the learning rate of local device, and $\mathbf{w}_i^{t,j}$ is the model of j-th local iteration of device i training with global model $\mathbf{w}^t$.

When a local device i has finished its k_i local updates to train the global model $\mathbf{w}^t$, it computes the overall gradient $\mathbf{g}_i^t$ in local training, that is:

$$\mathbf{g}_i^t = \mathbf{w}_i^{t,0} - \mathbf{w}_i^{t,k_i}. \tag{4}$$

Note that when a local device i receives the t-th global model $\mathbf{w}^t$, it will be initialized with $\mathbf{w}_i^{t,0} = \mathbf{w}^t$. Then, the local device will upload the compressed gradient to the PS. In this paper, we employ top_k sparsification algorithm [2] as the compressor and other compressors can also be employed.

The compression rate of the top_k compression operator is defined as $\delta = \frac{k}{d}$, where k represents the number of parameters retained after compression, and d denotes the total number of original parameters. After compressed by top_k compressor, the device i will send the compressed gradient $\tilde{\mathbf{g}}_i^t = C_{\delta_i}(\mathbf{g}_i^t)$ to the parameter server. We then use C_{δ_i} to express the top_k compressor with a compression rate of δ_i. The amount of data transmitted is proportional to the compression rate, so the time for transmitting a compressed gradient is also proportional to the compression rate.

Considering the diversity in computational capabilities and communication capabilities among devices, the total time for training and communication of device i is:

$$d_i = k_i\alpha_i + \delta_i\beta_i, \tag{5}$$

where the local updating frequency is k_i and the compression rate is δ_i. We define C_{δ_i} as the compressor with compression rate δ_i. Let α_i denote the computation time required for one local update on device i, and β_i represent the communication time for transmitting a full-size gradient on device i. Given that the download bandwidth is typically higher than the upload bandwidth [16,33], our attention is primarily directed toward the communication time involved in transmitting the models from devices to the PS during the model exchange process.

During local training and communication, the server continuously receives gradients from local devices. We define $\mathbf{S}^t$ as the set of local devices to which the server has received gradients in the t-th global round. The PS then aggregates the received local gradients from $\mathbf{S}^t$ and updates the global model:

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \frac{\eta_g}{|\mathbf{S}^t|} \sum_{i \in \mathbf{S}^t} \tilde{\mathbf{g}}_i^{t-\tau_i^t}, \tag{6}$$

where η_g is the global learning rate. Due to the asynchronous nature, the gradient may be stale. That is, the gradient of device $\tilde{\mathbf{g}}_i^{t-\tau_i^t}$ is generated by device i to train the global model $\mathbf{w}^{t-\tau_i^t}$. When the PS receives $\tilde{\mathbf{g}}_i^{t-\tau_i^t}$, it is executing the t-th round of aggregation. Building on the preceding analysis, the staleness of device i to train the global model $\mathbf{w}^{t-\tau_i^t}$ is τ_i^t that is defined as:

$$\tau_i^t = t - \max_{t' < t} \{t' | i \in \mathbf{S}^{t'}\}. \tag{7}$$

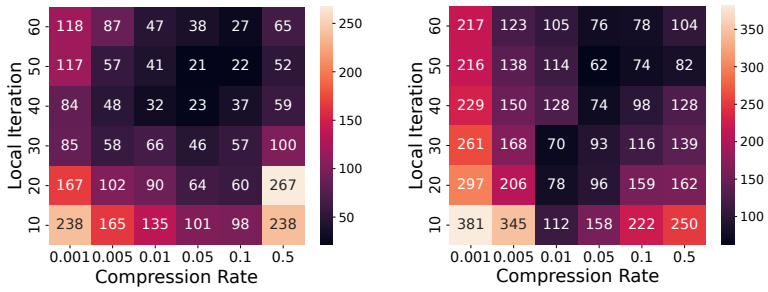
According to the characteristics of AFL with periodic aggregation, the time interval between every two global rounds is fixed. Therefore, to employ the staleness τ_i^t in the theoretical analysis, τ_i^t can be also calculated as $\lceil \frac{d_i}{\tilde{T}} \rceil$, where $\tilde{T}$ is the duration of one round. This equivalence holds when the values of local updating frequency and compression rate are constant for each device.

3 The Joint Approach to Local Updating and Gradient Compression

In this section, we first explore the impacts of both local updating frequency and compression rate, providing two motivation experiments to illustrate the dilemma discussed in Sec. 1. Then, we derive the upper bound of the convergence rate through theoretical analysis, leading to the proposal of FedLuck, an AFL framework that jointly and adaptively decide the local updating frequency and the compression rate for enhanced training efficiency.

3.1 Illustrative Examples of Motivation

As mentioned in Sec. 1, model staleness in AFL is affected by the time each device takes in one round, comprising local updating and communication time. We regulate local updating time with the local updating frequency and communication time with the compression rate. Our exploration reveals optimal values for both parameters, as elaborated below.



(a) CNN@FMNIST (b) LSTM@SC

Fig. 1: The minimum global rounds required to reach the target accuracy for the global model under different compression rates and local updating frequencies.

We conducted both image classification (training CNN on Fashion-MNIST dataset [31], abbreviated as FMNIST) and speech recognition (training LSTM on Speech Commands dataset [27], abbreviated as SC) experiments, which is sufficient to validate our motivation. We used 5 devices and a PS for both experiments. We set the local updating frequency to integers multiple of ten between 10 and 60, and the compression rate to 0.001, 0.005, along with 10 and 100 times these values. The detailed results are depicted in Fig. 1 which shows the minimum number of global rounds required for the global model to achieve an accuracy of 0.85 and 0.5. Specifically, deviating from the experimentally determined efficient settings either by increasing or decreasing can lead to a convergence process that is up to 3 times slower than the rate achieved at the fastest observed settings.

3.2 Our Proposed AFL Framework: FedLuck

Theoretical Analysis. Once we have confirmed our motivation, we must address the following problem: Given a specific AFL system, how can we jointly and adaptively decide the local updating frequency and the compression rate to converge efficiently?

We delve into the examination of the convergence bound of the global loss function over T global iterations. To commence, we outline the assumptions imposed on the training model, which are commonly embraced in pioneering federated learning works [14] and similar to [22].

Assumption 1. (Bounded local variance). *There exists a constant σ, such that the variance of each local estimator is bounded by:*

$$\mathbb{E}_{\xi \sim D_i}\left[||\nabla F_i(\mathbf{w}, \xi) - \nabla F_i(\mathbf{w})||\right] \leq \sigma, \forall i \in \mathcal{N}, \forall \mathbf{w} \in \mathbb{R}^d. \tag{8}$$

Assumption 2. (Bounded function heterogeneity). *There exist N constants $\zeta_i^2 \geq 0, i \in \{1, 2, \ldots, N\}$, such that the variance of the model gradients is bounded by:*

$$||\nabla F_i(\mathbf{w}) - \nabla F(\mathbf{w})||^2 \leq \zeta_i^2, \forall \mathbf{w} \in \mathbb{R}^d, \tag{9}$$

and we define $\zeta^2 := \frac{1}{N}\sum_{i \in \mathcal{N}} \zeta_i^2$.

Assumption 3. (L-smooth). *The loss functions F and F_i are L-smooth with a constant $L \geq 0$ such that:*

$$||\nabla F_i(\mathbf{y}) - \nabla F_i(\mathbf{x})|| \leq L||\mathbf{y} - \mathbf{x}||, \forall \mathbf{x}, \mathbf{y} \in \mathbb{R}^d. \tag{10}$$

Assumption 4. (Bounded gradient). *There exists a constant $G \geq 0$ such that the norm of local gradient is bounded by:*

$$||\nabla F_i(\mathbf{w})||^2 \leq G^2, \forall \mathbf{w} \in \mathbb{R}^d. \tag{11}$$

These assumptions are widely adopted and considered standard in the context of non-convex optimization problems. We show the detailed theoretical analysis in https://github.com/Aegsteh-T/FedLuck_Euro_Par.

Theorem 1. If all local models are initialized by the same weight $\mathbf{w}^0$. Assuming the global learning rate and local learning rate $\eta_g, \eta_l \leq \frac{1}{L}$ and $Q_l = 1 - 6(1 - \eta_l k)^2 \geq 0$, the mean square of gradient after T global iterations is bounded as follows:

$$\begin{aligned} \frac{1}{T}\sum_{t=0}^{T-1} ||\nabla F(\mathbf{w}^t)||^2 \leq & \frac{2[F(\mathbf{w}^0) - F(\mathbf{w}^*)]}{\eta_g Q_l T} \\ & + \frac{\eta_l^2 k^2([(1-\delta)B_1 + B_2]}{Q_l} + \frac{\eta_l^2 k^4 [2\tau_{max}^2(2-\delta) + 1]B_1}{Q_l}, \end{aligned} \tag{12}$$

where $B_1 = 18(\sigma^2 + \zeta^2 + G^2)$ and $B_2 = 6(\sigma^2 + \zeta^2)$ represent the bias terms in the system. $\tau_{max} = \max_{i \in \mathcal{N}} \tau_i$ is the max model staleness of all devices.

Corollary 1. Choosing global learning rate $\eta_g = \mathcal{O}(\sqrt{\frac{k}{T}})$ and local learning rate $\eta_l = \mathcal{O}(T^{-1/4}k^{-5/2}\delta^{-1/4})$, we have the convergence rate:

$$\begin{aligned} \frac{1}{T}\sum_{t=0}^{T-1} ||\nabla F(\mathbf{w}^t)||^2 \leq & \mathcal{O}(\frac{F^*}{\sqrt{kT}}) + \mathcal{O}(\frac{(1-\delta)B_1 + B_2}{k^3\sqrt{T\delta}}) \\ & + \mathcal{O}(\frac{(\tau_{max}^2(2-\delta) + 1)B_1}{k\sqrt{T\delta}}). \end{aligned} \tag{13}$$

Given that the third term exerts the highest order of influence on the convergence speed, our focus is directed toward this term to discuss strategies for achieving efficient convergence. We name the part related to local updating frequency and compression rate in the third term as **key convergence factor**. To get the fastest convergence, we should minimize the key convergence factor. According to the definition of staleness, we expand $\tau_{max} = \frac{k\alpha + \delta\beta}{\tilde{T}}$, where $\alpha = \alpha_m, \beta = \beta_m, k = k_m, \delta = \delta_m, m = \arg\max_i \tau_i$. Then we get the key convergence factor as:

$$\phi(k, \delta) = \frac{(k\alpha + \delta\beta)^2(2-\delta) + \tilde{T}^2}{\tilde{T}^2 k\sqrt{\delta}}. \tag{14}$$

To extend the key convergence factor to all devices, we aim to find the optimal solutions of local updating frequency k_i and compression rate δ_i for each device i

to achieve fast convergence. This optimization problem is formulated to minimize the key convergence factor $\phi(k_i, \delta_i)$ while considering constraints on k_i and δ_i within specific ranges. By solving this heuristic optimization problem, we obtain adaptive values for k_i and δ_i for each device, enabling efficient convergence in the training process. The optimization problem is formulated as follows:

$$\begin{aligned} &\min_{k_i, \delta_i} \ \phi(k_i, \delta_i) \\ &s.t.\ k_i \in [k_{min}, k_{max}], \quad \delta_i \in [\delta_{min}, \delta_{max}], \end{aligned} \tag{15}$$

where k_{min} and k_{max} are the minimal local updating frequency and maximal local updating frequency, respectively. δ_{min} and δ_{max} are the similar meanings for compression rates. Based on the formula (Eq. 15), we can find the adaptive

Algorithm 1 FedLuck

Server:
Receive α_i from devices
Test β_i with devices
Get the optimal k_i and δ_i according to Eq. 15
Send (k_i, δ_i) to device i
Broadcast $\mathbf{w}^0$ to all devices and start them
for $t = 0, 1, \cdots, T-1$: **do**
  Continuously receive $\tilde{\mathbf{g}}_i^{t-\tau_i^t}$ from local devices set $\mathbf{S}^t$
  $\mathbf{w}^{t+1} \leftarrow \mathbf{w}^t - \frac{\eta_g}{|\mathbf{S}^t|} \sum_{i \in \mathbf{S}^t} \tilde{\mathbf{g}}_i^{t-\tau_i^t}$
  for $i \in \mathbf{S}^t$ **do**
    Send new global model $\mathbf{w}^{t+1}$ to i
  end for
end for
Notify all devices to *STOP*

Device:
Test α_i and send it to the server
Test β_i with the server
Receive (k_i, δ_i) from server
while not *STOP* **do**
  Receive $\mathbf{w}^t$ from server
  Set $\mathbf{w}_i^{t,0} \leftarrow \mathbf{w}^t$
  for each local iteration $j \in 0, 1, \cdots, k_i - 1$: **do**
    $\mathbf{w}_i^{t,j+1} \leftarrow \mathbf{w}_i^{t,j} - \eta_l \nabla F_i(\mathbf{w}_i^{t,j}; \xi_i^{t,j})$
  end for
  Compute gradient $\mathbf{g}_i^t \leftarrow \mathbf{w}_i^{t,0} - \mathbf{w}_i^{t,k_i}$
  Compute compressed gradient $\tilde{\mathbf{g}}_i^t \leftarrow C_{\delta_i}(\mathbf{g}_i^t)$
  Send $\tilde{\mathbf{g}}_i^t$ to server
end while

values k_i and δ_i for each device i. We show FedLuck in Alg. 1. Devices must determine their local updating frequency and compression rate before training by conducting tests on local updating and communication time. Local training rounds are performed on each device to calculate the average local training time per iteration, denoted as α_i. Devices also measure gradient transmission time by sending compressed gradients with varying timestamps and compression rates, allowing the server to compute the communication time required for a full gradient, denoted as β_i. The server then solves the optimization problem (Eq. 15) to find the values for k_i and δ_i, which are sent to each device. Devices apply these parameters for AFL. A comparison of our approach with other baselines is presented in the subsequent section.

4 Experiments

In this section, we compare our proposed method, FedLuck, with four baselines. We outline the experimental settings, encompassing the tasks, baselines, and

metrics. We also detail the hardware and hyper-parameter configurations used in our experiments. Finally, we present the results under empirical settings.

4.1 Experiment Tasks

Tasks. In our experiments, we evaluated the performance of our FedLuck framework using three diverse tasks: (1) a four-layer CNN model for FMNIST (Fashion-MNIST [31]) dataset for image classification, inspired by [17], (2) VGG11s [25] on CIFAR-10[4] for image classification. VGG11s' architecture is similar to [24], and (3) an LSTM for SC (Speech Commands, [27]). The LSTM features two hidden layers with 128 units each. These models are tested across both visual and auditory domains to demonstrate the versatility of our approach.

4.2 Baselines and Metrics

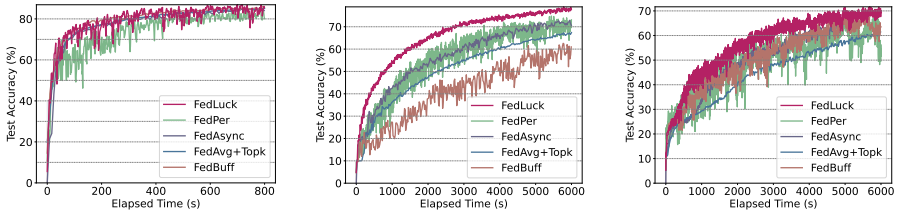
Baselines. We selected four classical algorithms as baselines for performance comparison. (1) **FedPer:** An AFL method with periodic aggregation, maintaining fixed and uniform local updating frequency and consistent compression rate across devices [13]. (2) **FedBuff:** An AFL method that aggregates a fixed number of gradients in each global round [22]. (3) **FedAsync:** FedAsync [32] is an AFL method that incorporates a regularization term to enhance performance when dealing with staleness. Moreover, FedAsync changes fusion weight with respect to model staleness. (4) **FedAvg + Topk:** Based on the FedAvg [21] algorithm, we have incorporated the Topk gradient compression technique to mitigate communication overhead [2].

Metrics. We assess and compare the performance of FedLuck with the other baselines using the following metrics. (1) **Test accuracy** is the ratio of correct predictions to the total test data points. (2) **Communication consumption** measures the data volume exchanged in federated learning, assessing communication efficiency.

4.3 Simulation Experiments

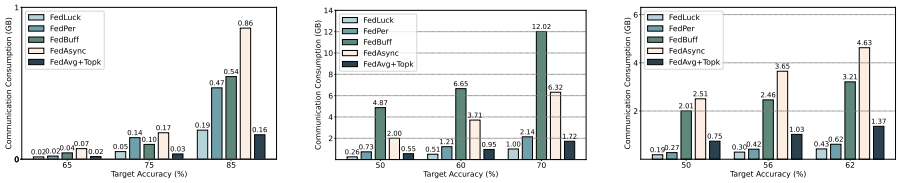
Simulation Settings. We conducted simulation experiments to evaluate the performance of FedLuck on a server running Ubuntu 18.04.6 LTS, equipped with an Intel(R) Xeon(R) Silver 4210 CPU (20 cores, 40 threads) and 4 Nvidia GeForce GTX 3090 GPUs with CUDA version 11.2. We implemented the model training for each device on PyTorch, with Python version 3.8.13 and Torch version 1.12.0. We consider a common scenario where devices communicate with the central PS through a low-bandwidth network. There are 10 devices and 1 PS in total. Our simulation focuses on a typical scenario where each device's bandwidth ranges from 0.25 Mb/s to 2 Mb/s [21]. In addition, to simulate the heterogeneity of computation, we assume that the computation time of a local iteration of a simulated device follows a uniform distribution. The minimum value of the uniform distribution is multiple times of the computation time of one local iteration on the Nvidia GeForce GTX 3090 24GB GPU. The maximum value of the uniform distribution is four times the minimum value.

[4] http://www.cs.toronto.edu/~kriz/cifar.html



(a) CNN@FMNIST (b) VGG11s@CIFAR-10 (c) LSTM@SC

Fig. 2: Test Accuracy and Elapsed Time comparison between FedLuck and four baselines on three tasks in IID setting. FedLuck reduces training time by 55% in average compared with baselines.



(a) CNN@FMNIST (b) VGG11s@CIFAR-10 (c) LSTM@SC

Fig. 3: Communication consumption between FedLuck and four baselines at target accuracy on three tasks in IID setting. FedLuck reduces communication consumption by 56% in average compared with baselines.

We utilized the momentum-SGD optimizer across all three tasks, initializing the learning rates to 0.01 for the CNN model on FMNIST and VGG11s on CIFAR-10, and 0.016 for the LSTM model on SC, with a momentum of 0.9 for all models. A batch size of 64 was employed for the LSTM and CNN models, while a batch size of 32 was used for the VGG model, to efficiently process the training data. These settings were carefully selected to enhance the training process and ensure reliability during evaluation.

Simulation Results. We evaluated the efficiency of FedLuck in terms of convergence speed and communication consumption. The horizontal axis represents the training time, while the vertical axis represents the test accuracy of the global model on the test dataset. This result highlights the effectiveness of jointly optimizing local updating frequencies and gradient compression rates for heterogeneous devices. As shown in Fig. 2c, FedLuck takes 2517.61s to reach 62% accuracy for LSTM on SC, while FedPer, FedBuff, FedAsync, FedAvg+Topk requires 3438.00s, 3452.12s, 3887.05s, 6136.02s, respectively. These timings translate to training speedups of 1.36×, 1.37×, 1.54×, 2.44× for FedLuck compared to FedPer, FedBuff, FedAsync, FedAvg+Topk.

To illustrate the communication efficiency of FedLuck, we present the network communication consumption for different algorithms as they attain varying target accuracies, detailed in Fig. 3. FedLuck demonstrates the most efficient network communication consumption in 6 out of 9 test cases. Even in the remaining three cases where it's not the best, FedLuck's performance is notably close to the best result. In Fig. 3b for VGG11s on CIFAR-10, when achieving the

Table 1: Comparison of elapsed time and communication consumption for reaching the target accuracy in Non-IID setting.

Model@Dataset	Target Accuracy(%)	Method	Elapsed Time(s)	Communication Consumption(GB)
CNN@FMNIST	65	FedLuck	**41.92**	**0.02**
		FedPer	64.44	0.05
		FedBuff	57.32	0.06
		FedAsync	66.66	0.07
		FedAvg+Topk	66.65	**0.02**
	75	FedLuck	**77.92**	**0.04**
		FedPer	211.43	0.12
		FedBuff	141.17	0.13
		FedAsync	227.64	0.24
		FedAvg+Topk	199.95	**0.04**
	85	FedLuck	**581.92**	0.32
		FedPer	644.94	0.36
		FedBuff	614.88	0.53
		FedAsync	841.53	0.87
		FedAvg+Topk	1066.39	**0.21**
VGG11s@CIFAR-10	50	FedLuck	**837.00**	**0.31**
		FedPer	1257.26	0.76
		FedBuff	3694.49	5.13
		FedAsync	2134.37	2.81
		FedAvg+Topk	2677.70	0.65
	60	FedLuck	**1484.99**	**0.55**
		FedPer	1959.25	1.18
		FedBuff	5979.43	8.28
		FedAsync	3163.80	4.16
		FedAvg+Topk	4471.76	1.08
	65	FedLuck	**1965.00**	**0.73**
		FedPer	2760.26	1.66
		FedBuff	8635.33	11.94
		FedAsync	4565.33	6.00
		FedAvg+Topk	5944.50	1.44
LSTM@SC	50	FedLuck	**1438.16**	0.54
		FedPer	1754.43	**0.36**
		FedBuff	1586.80	1.48
		FedAsync	2007.81	2.39
		FedAvg+Topk	2813.85	0.63
	60	FedLuck	**2436.00**	0.91
		FedPer	3619.29	**0.73**
		FedBuff	3176.19	2.95
		FedAsync	3124.58	3.72
		FedAvg+Topk	4592.93	1.02
	63	FedLuck	**2705.98**	1.01
		FedPer	4062.71	**0.82**
		FedBuff	4418.68	4.11
		FedAsync	3604.80	4.30
		FedAvg+Topk	5409.86	1.20

70% target accuracy, FedLuck, FedPer, FedBuff, FedAsync and FedAvg+Topk are 1.00GB, 2.14GB, 12.02GB, 6.32GB and 1.72GB, respectively. Concretely, FedLuck can save network communication consumption by 53%, 92%, 84%, and 41% for VGG11s on CIFAR-10, compared to the respective baselines (FedPer, FedBuff, FedAsync, FedAvg+Topk).

The impact of Non-IID data. In the experiments conducted above, we utilized only independent and identically distributed (IID) data. However, in federated learning, dealing with not identically and independently distributed (Non-IID) data is a critical concern that can decelerate convergence speed. We further conducted tests on the impact of Non-IID data, in line with Assumption 2 (Eq. 9). We generated Non-IID data using the Dirichlet distribution, randomly assigning it to each local device to create Non-IID data distributions across devices, with a concentration degree α of 1.0 [12], reflecting real-world scenarios. Tab. 1 shows the results of our experiments with non-IID data. FedLuck continues to outpace the baselines in both convergence speed and communication consumption. For instance, FedLuck only uses 1965.00s to achieve the 65% target testing accuracy on VGG11s training on CIFAR-10, while FedPer, FedBuff, FedAsync and FedAvg+Topk use 2760.26s, 8635.33s, 4565.33s, 5944.50s, respec-

Table 2: Comparison of the top-1 test accuracy between FedLuck and methods that optimize only one factor over 800s for CNN, 6000s for VGG11s and LSTM.

Top-1 Accuracy	CNN@FMNIST	VGG11s@CIFAR-10	LSTM@SC
FedLuck	$\mathbf{86.9 \pm 0.4}\%$	$\mathbf{78.9 \pm 0.2}\%$	$\mathbf{71.0 \pm 0.4}\%$
Opt. CR	$83.2 \pm 0.5\%$	$67.6 \pm 0.4\%$	$70.8 \pm 0.5\%$
Opt. LF	$81.5 \pm 0.4\%$	$77.0 \pm 0.3\%$	$69.8 \pm 0.3\%$

tively. FedLuck not only converged to 65% target testing accuracy in the shortest possible time but also required less communication consumption.

The necessity of joint optimization. To explore the essential "joint" characteristic of our FedLuck method, we conducted experiments and compared it with two separate baselines. In the first baseline Opt. CR, we adopted a fixed gradient compression rate and optimized only the local updating frequency. Conversely, in the second baseline Opt. LF, we kept the local updating frequency static and focused solely on optimizing the gradient compression rate. According to Tab. 2, FedLuck excels in all three tasks, demonstrating that the joint optimization of both local updating frequency and compression rate is beneficial in creating a more efficient AFL system.

5 Conclusion

In this paper, we tackle the inherent challenges of AFL arising from device heterogeneity and low-bandwidth communication environments, focusing on the issue of model staleness. We propose FedLuck, a novel AFL framework to achieve enhanced training efficiency by jointly and adaptively deciding the local updating frequency and compression rate. Through a thorough exploration of these factors, we derive a convergence upper bound directly linked to local update frequency and gradient compression rate, which allows us to carefully adjust these parameters to optimize the process. FedLuck demonstrates impressive results, significantly reducing communication consumption by 56% and training time by approximately 55%, making it a competitive alternative compared to existing baselines in scenarios with heterogeneous and low-bandwidth conditions. To our best knowledge, we are the first to jointly optimize AFL in this manner, thereby contributing to more efficient and effective federated learning in real-world settings. The insights and contributions presented in our paper pave the way for future research to refine and extend the approach, addressing other challenges in AFL and advancing the state-of-the-art in the field.

ACKNOWLEDGMENT

This work is supported in part by the National Key Research and Development Project of China (Grant No. 2023YFF0905502), Shenzhen Science and Technology Program (Grant No. JCYJ20220818101014030).

References

1. Abdelmoniem, A.M., Canini, M.: Dc2: Delay-aware compression control for distributed machine learning. In: IEEE INFOCOM 2021-IEEE Conference on Com-

puter Communications. pp. 1–10. IEEE (2021)
2. Aji, A.F., Heafield, K.: Sparse communication for distributed gradient descent. arXiv preprint arXiv:1704.05021 (2017)
3. Alistarh, D., Grubic, D., Li, J., Tomioka, R., Vojnovic, M.: Qsgd: Communication-efficient sgd via gradient quantization and encoding. Advances in neural information processing systems **30** (2017)
4. Bernstein, J., Wang, Y.X., Azizzadenesheli, K., Anandkumar, A.: signsgd: Compressed optimisation for non-convex problems. In: International Conference on Machine Learning. pp. 560–569. PMLR (2018)
5. Chai, Z., Chen, Y., Anwar, A., Zhao, L., Cheng, Y., Rangwala, H.: Fedat: A high-performance and communication-efficient federated learning system with asynchronous tiers. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. pp. 1–16 (2021)
6. Chen, M., Mao, B., Ma, T.: Efficient and robust asynchronous federated learning with stragglers. In: International Conference on Learning Representations (2019)
7. Chen, M., Mao, B., Ma, T.: Fedsa: A staleness-aware asynchronous federated learning algorithm with non-iid data. Future Generation Computer Systems **120**, 1–12 (2021)
8. Chen, Y., Ning, Y., Slawski, M., Rangwala, H.: Asynchronous online federated learning for edge devices with non-iid data. In: 2020 IEEE International Conference on Big Data (Big Data). pp. 15–24. IEEE (2020)
9. Cui, L., Su, X., Zhou, Y., Liu, J.: Optimal rate adaption in federated learning with compressed communications. In: IEEE INFOCOM 2022-IEEE Conference on Computer Communications. pp. 1459–1468. IEEE (2022)
10. van Dijk, M., Nguyen, N.V., Nguyen, T.N., Nguyen, L.M., Tran-Dinh, Q., Nguyen, P.H.: Asynchronous federated learning with reduced number of rounds and with differential privacy from less aggregated gaussian noise. arXiv preprint arXiv:2007.09208 (2020)
11. Hsieh, K., Harlap, A., Vijaykumar, N., Konomis, D., Ganger, G.R., Gibbons, P.B., Mutlu, O.: Gaia:{Geo-Distributed} machine learning approaching {LAN} speeds. In: 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17). pp. 629–647 (2017)
12. Hsu, T.M.H., Qi, H., Brown, M.: Measuring the effects of non-identical data distribution for federated visual classification. arXiv preprint arXiv:1909.06335 (2019)
13. Hu, C.H., Chen, Z., Larsson, E.G.: Scheduling and aggregation design for asynchronous federated learning over wireless networks. IEEE Journal on Selected Areas in Communications **41**(4), 874–886 (2023)
14. Koloskova, A., Stich, S.U., Jaggi, M.: Sharper convergence guarantees for asynchronous sgd for distributed and federated learning. Advances in Neural Information Processing Systems **35**, 17202–17215 (2022)
15. Konečnỳ, J., McMahan, H.B., Ramage, D., Richtárik, P.: Federated optimization: Distributed machine learning for on-device intelligence. arXiv preprint arXiv:1610.02527 (2016)
16. Konečnỳ, J., McMahan, H.B., Yu, F.X., Richtárik, P., Suresh, A.T., Bacon, D.: Federated learning: Strategies for improving communication efficiency. arXiv preprint arXiv:1610.05492 (2016)
17. Li, T., Sahu, A.K., Talwalkar, A., Smith, V.: Federated learning: Challenges, methods, and future directions. IEEE signal processing magazine **37**(3), 50–60 (2020)
18. Lian, X., Huang, Y., Li, Y., Liu, J.: Asynchronous parallel stochastic gradient for nonconvex optimization. Advances in neural information processing systems **28** (2015)

19. Lin, Y., Han, S., Mao, H., Wang, Y., Dally, W.J.: Deep gradient compression: Reducing the communication bandwidth for distributed training. arXiv preprint arXiv:1712.01887 (2017)
20. Lu, R., Song, J., Chen, B., Cui, L., Wang, Z.: Dagc: Data-aware adaptive gradient compression
21. McMahan, B., Moore, E., Ramage, D., Hampson, S., y Arcas, B.A.: Communication-efficient learning of deep networks from decentralized data. In: Artificial intelligence and statistics. pp. 1273–1282. PMLR (2017)
22. Nguyen, J., Malik, K., Zhan, H., Yousefpour, A., Rabbat, M., Malek, M., Huba, D.: Federated learning with buffered asynchronous aggregation. In: International Conference on Artificial Intelligence and Statistics. pp. 3581–3607. PMLR (2022)
23. Nori, M.K., Yun, S., Kim, I.M.: Fast federated learning by balancing communication trade-offs. IEEE Transactions on Communications **69**(8), 5168–5182 (2021)
24. Sattler, F., Wiedemann, S., Müller, K.R., Samek, W.: Robust and communication-efficient federated learning from non-iid data. IEEE transactions on neural networks and learning systems **31**(9), 3400–3413 (2019)
25. Simonyan, K., Zisserman, A.: Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556 (2014)
26. Wang, Z., Zhang, Z., Tian, Y., Yang, Q., Shan, H., Wang, W., Quek, T.Q.: Asynchronous federated learning over wireless communication networks. IEEE Transactions on Wireless Communications **21**(9), 6961–6978 (2022)
27. Warden, P.: Speech commands: A dataset for limited-vocabulary speech recognition. arXiv preprint arXiv:1804.03209 (2018)
28. Wen, W., Xu, C., Yan, F., Wu, C., Wang, Y., Chen, Y., Li, H.: Terngrad: Ternary gradients to reduce communication in distributed deep learning. Advances in neural information processing systems **30** (2017)
29. Wu, D., Ullah, R., Rodgers, P., Kilpatrick, P., Spence, I., Varghese, B.: Communication efficient dnn partitioning-based federated learning. arXiv preprint arXiv:2304.05495 (2023)
30. Wu, W., He, L., Lin, W., Mao, R., Maple, C., Jarvis, S.: Safa: A semi-asynchronous protocol for fast federated learning with low overhead. IEEE Transactions on Computers **70**(5), 655–668 (2020)
31. Xiao, H., Rasul, K., Vollgraf, R.: Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. arXiv preprint arXiv:1708.07747 (2017)
32. Xie, C., Koyejo, S., Gupta, I.: Asynchronous federated optimization. arXiv preprint arXiv:1903.03934 (2019)
33. Xu, Y., Liao, Y., Xu, H., Ma, Z., Wang, L., Liu, J.: Adaptive control of local updating and model compression for efficient federated learning. IEEE Transactions on Mobile Computing (2022)
34. Zhang, W., Gupta, S., Lian, X., Liu, J.: Staleness-aware async-sgd for distributed deep learning. arXiv preprint arXiv:1511.05950 (2015)
35. Zhang, Y., Liu, D., Duan, M., Li, L., Chen, X., Ren, A., Tan, Y., Wang, C.: Fedmds: An efficient model discrepancy-aware semi-asynchronous clustered federated learning framework. IEEE Transactions on Parallel and Distributed Systems **34**(3), 1007–1019 (2023)
36. Zheng, S., Meng, Q., Wang, T., Chen, W., Yu, N., Ma, Z.M., Liu, T.Y.: Asynchronous stochastic gradient descent with delay compensation. In: International Conference on Machine Learning. pp. 4120–4129. PMLR (2017)
37. Zhu, F., Hao, J., Chen, Z., Zhao, Y., Chen, B., Tan, X.: Stafl: Staleness-tolerant asynchronous federated learning on non-iid dataset. Electronics **11**(3), 314 (2022)